Матрицы, таблицы и факторы в R

Подготовил Давид Кристофер

Матрицы

ОДИН ЗАКАЗ — ОДНА СТРОКА

Лаборатория оформляет два заказа расходных материалов:

Заказ	Наконечники	Пробирки	Перчатки
A	2	1	0
B	1	0	3

- Наблюдения обычно располагают по строкам, а признаки — по столбцам;
- Это соответствует принципам аккуратных данных (tidy data);
- Здесь наблюдение — заказ, а признаки — количества упаковок каждого материала.

СОБЕРЁМ КОЛИЧЕСТВА В МАТРИЦУ

Матрица — двумерная структура данных с элементами одного типа.

```
Q <- matrix(                          # вызываем matrix()
  c(2, 1, 0, 1, 0, 3),                # количества упаковок
  nrow = 2,                           # число строк
  byrow = TRUE,                       # заполняем по строкам
  dimnames = list(                    # имена строк и столбцов
    c("A", "B"),                      # строки: заказы
    c("tips", "tubes", "gloves")      # столбцы: материалы
  )
)
Q                                     # выводим матрицу
```

Число столбцов определяется автоматически: 6 / 2 = 3.

По умолчанию — по столбцам, `byrow = TRUE` — по строкам.

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

В КАКОМ ПОРЯДКЕ ЗАПОЛНЯЮТСЯ ЯЧЕЙКИ?

По умолчанию `matrix()` заполняет столбцы сверху вниз.
При `byrow = TRUE` матрица заполняется по строкам, слева направо.

```
matrix(1:6, nrow = 2)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
matrix(1:6, nrow = 2, byrow = TRUE)
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

ИМЕНА СТРОК И СТОЛБЦОВ

Имена можно получить и изменить после создания матрицы.

```
colnames(Q)

rownames(Q)
```

```
[1] "tips"   "tubes"  "gloves"

[1] "A" "B"
```

```
colnames(Q) <- c("tip", "tube", "glove")
rownames(Q) <- c("ordA", "ordB")

Q
```

```
     tip tube glove
ordA   2    1     0
ordB   1    0     3
```

МАТРИЦА КАК НАБОР СТРОК ИЛИ СТОЛБЦОВ

`cbind()` соединяет векторы в столбцы, `rbind()` — в строки.

По столбцам

```
Q <- cbind(
  tips = c(2, 1),
  tubes = c(1, 0),
  gloves = c(0, 3)
)
rownames(Q) <- c("A", "B")
Q
```

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

По строкам

```
Q <- rbind(
  A = c(2, 1, 0),
  B = c(1, 0, 3)
)
colnames(Q) <-
  c("tips", "tubes", "gloves")
Q
```

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

СОЗДАНИЕ МАТРИЦЫ ЧЕРЕЗ `dim()`

```r
Q <- c(2, 1, 1, 0, 0, 3)
class(Q)
```

```
[1] "numeric"
```

```r
dim(Q) <- c(2, 3)  # заполняем по столбцам
class(Q)
```

```
[1] "matrix" "array"
```

```r
dimnames(Q) <- list(
  c("A", "B"),
  c("tips", "tubes", "gloves")
)
Q
```

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

ВЫБОР ЯЧЕЙКИ МАТРИЦЫ

Выбор: `Q[строка, столбец]`.
Выберем число упаковок пробирок в заказе A:

Q

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

```
dim(Q)
```

```
[1] 2 3
```

```
Q[1, 2]
```

```
[1] 1
```

```
Q["A", "tubes"]
```

```
[1] 1
```

МОЖНО ВЫБРАТЬ ВЕСЬ ЗАКАЗ ИЛИ МАТЕРИАЛ

Пустая координата означает «все».
С `drop = FALSE` результат остаётся матрицей.

Q

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

```
class(Q[, "gloves"])
class(Q[, "gloves",
        drop = FALSE])
```

```
[1] "numeric"
[1] "matrix" "array"
```

```
# состав заказа B
Q["B", ]
```

```
# перчатки: вектор
Q[, "gloves"]
```

```
# перчатки: матрица
Q[, "gloves",
  drop = FALSE]
```

```
  tips  tubes gloves
     1      0      3
```

```
A B
0 3
```

```
  gloves
A      0
B      3
```

ИНДЕКСЫ СТРОК И СТОЛБЦОВ

```
X <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)
```

X

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
# строки 1 и 2, столбцы 2 и 3
X[c(1, 2), c(2, 3)]

# строки 3 и 2 целиком, в этом порядке
X[c(3, 2), ]

# вся матрица
X[, ]

# все строки, кроме первой
X[-1, ]
```

`rowSums()` и `colSums()`

Сумма по строке — сколько упаковок в заказе.
Сумма по столбцу — сколько упаковок одного материала нужно купить.

Q

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

```
rowSums(Q)
```

```
A B
3 4
```

```
colSums(Q)
```

```
  tips  tubes gloves
     3      1      3
```

Суммировать значения в строке можно, потому что во всех столбцах указано количество упаковок.

ФУНКЦИЯ ДЛЯ СТРОК И СТОЛБЦОВ: `apply()`

Найдём максимум в каждой строке и каждом столбце.
Второй аргумент: `1` — строки, `2` — столбцы.

Q

```
   tips tubes gloves
A     2     1      0
B     1     0      3
```

```
# максимум в строке
apply(Q, 1, max)
```

```
A B
2 3
```

```
# максимум в столбце
apply(Q, 2, max)
```

```
  tips  tubes gloves
     2      1      3
```

Передаём `max` без скобок: `apply()` вызывает её для каждой строки или столбца.

УМНОЖЕНИЕ МАТРИЦЫ НА СКАЛЯР

Умножение на скаляр выполняется поэлементно.
Увеличим каждый заказ вдвое:

Q

```
   tips tubes gloves
A    2     1      0
B    1     0      3
```

```
Q * 2
```

Результат

```
   tips tubes gloves
A    4     2      0
B    2     0      6
```

Результат — новая матрица. Исходная `Q` не меняется без присваивания.

СТОИМОСТЬ ЗАКАЗОВ: МАТРИЧНАЯ ЗАПИСЬ

Для каждого заказа складываем произведения количества упаковок на цену. Вектор цен: 120, 80 и 150 руб. — в порядке столбцов `Q`.

$$c = Qp = \underbrace{\begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & 3 \end{pmatrix}}_{Q:\ 2\times 3} \underbrace{\begin{pmatrix} 120 \\ 80 \\ 150 \end{pmatrix}}_{p:\ 3\times 1} = \begin{pmatrix} 2\cdot 120 + 1\cdot 80 + 0\cdot 150 \\ 1\cdot 120 + 0\cdot 80 + 3\cdot 150 \end{pmatrix} = \underbrace{\begin{pmatrix} 320 \\ 570 \end{pmatrix}}_{c:\ 2\times 1}$$

```
price <- c(120, 80, 150)
Q %*% price
```

```
  [,1]
A  320
B  570
```

`%*%` — матричное умножение.

ТРАНСПОНИРОВАНИЕ И СРАВНЕНИЕ ОПЕРАЦИЙ

После транспонирования `t(Q)` столбцы — заказы, строки — материалы.

$$(Qp)^\mathsf{T} = p^\mathsf{T} Q^\mathsf{T} = \begin{pmatrix} 320 & 570 \end{pmatrix}$$

Код R	Результат	Смысл	Форма
`sum(Q)`	7	Всего упаковок	число
`rowSums(Q)`	A = 3; B = 4	Упаковок в каждом заказе	вектор
`t(price) %*% t(Q)`	$\begin{pmatrix} 320 & 570 \end{pmatrix}$	Стоимость заказов, руб.	матрица 1 × 2
`price * t(Q)` `# цены повторяются`	$\begin{pmatrix} 240 & 120 \\ 80 & 0 \\ 0 & 450 \end{pmatrix}$	Стоимость материалов, руб.	матрица 3 × 2
`colSums(price * t(Q))`	A = 320; B = 570	Стоимость заказов, руб.	вектор

`price` — вектор

```
[1] 120  80 150
```

Можно и `price %*% t(Q)`.

`Q`

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

`t(Q)`

```
        A B
tips    2 1
tubes   1 0
gloves  0 3
```

БОЛЬШЕ ДВУХ ИЗМЕРЕНИЙ: `array()`

Матрица двумерна; массив может иметь больше измерений.
Объединим исходный `Q` и удвоенный `2 * Q` в два слоя.

```
Q3 <- array(
  c(Q, 2 * Q),        # два варианта заказов
  dim = c(2, 3, 2)  # заказы, материалы, варианты
)
```

```
Q
```

```
  tips tubes gloves
A    2     1      0
B    1     0      3
```

```
Q3[, , 2]        # второй слой: удвоенные заказы
```

```
     [,1] [,2] [,3]
[1,]    4    2    0
[2,]    2    0    6
```

```
dim(Q3)          # размеры массива
Q3[1, 2, ]       # пробирки в A: оба варианта
```

```
[1] 2 3 2
[1] 1 2
```

ЧТО ДЕЛАТЬ С ДАННЫМИ РАЗНЫХ ТИПОВ?

В матрице один тип данных, в `data.frame` — свой тип для каждого столбца.

```
# все значения приводятся к тексту
M <- cbind(name = c("Lab A", "Lab B"),
           packs = rowSums(Q))
M
typeof(M)
```

```
  name    packs
A "Lab A" "3"
B "Lab B" "4"

[1] "character"
```

```
df <- data.frame(name = c("Lab A", "Lab B"),
                 packs = rowSums(Q))
df
typeof(df$name)
typeof(df$packs)
```

```
   name packs
A Lab A     3
B Lab B     4

[1] "character"
[1] "double"
```

Таблицы

СОЗДАНИЕ ТАБЛИЦЫ: `data.frame()`

Объединение векторов в таблицу:

```
# vector_1 и vector_2 уже созданы
df <- data.frame(vector_1, vector_2)
```

- Каждый вектор становится столбцом.
- Длина самого длинного вектора задаёт число строк.
- Передача одного вектора создаёт таблицу из одного столбца.

Если максимальная длина кратна остальным, R повторяет короткие векторы.
Иначе — ошибка. Например, длины 2 и 4 допустимы, а 3 и 4 — нет.

СОЗДАНИЕ ТАБЛИЦЫ ИЗ ГОТОВЫХ ВЕКТОРОВ

Сначала создадим векторы, затем передадим их в `data.frame()`.

```
rating <- 1:4                              # место в рейтинге
animal <- c("коала", "ёж", "ленивец", "панда")
country <- c("Австралия", "Италия", "Перу", "Китай")
avg_sleep_hours <- c(21, 18, 17, 10)  # среднее время сна, ч/сутки
super_sleepers <- data.frame(rating, animal, country, avg_sleep_hours)
super_sleepers
```

```
  rating  animal   country avg_sleep_hours
1      1   коала Австралия              21
2      2      ёж    Италия              18
3      3 ленивец      Перу              17
4      4   панда     Китай              10
```

СОЗДАНИЕ ВЕКТОРОВ ВНУТРИ ВЫЗОВА `data.frame()`

Имена аргументов становятся именами столбцов.

```
super_sleepers <- data.frame(
  rating = 1:4,                          # место в рейтинге
  animal = c("коала", "ёж", "ленивец", "панда"),
  country = c("Австралия", "Италия", "Перу", "Китай"),
  avg_sleep_hours = c(21, 18, 17, 10) # среднее время сна, ч/сутки
)
```

Структура и обзор таблицы

КЛАСС И СТРУКТУРА: `class()` И `str()`

`class()` показывает класс объекта, `str()` — его структуру и типы столбцов.

```
class(super_sleepers)
str(super_sleepers)
```

```
[1] "data.frame"

'data.frame':   4 obs. of  4 variables:
 $ rating         : int  1 2 3 4
 $ animal         : chr  "коала" "ёж" "ленивец" "панда"
 $ country        : chr  "Австралия" "Италия" "Перу" "Китай"
 $ avg_sleep_hours: num  21 18 17 10
```

`glimpse()`: АЛЬТЕРНАТИВА `str()`

Более наглядный вывод: каждый столбец показан в отдельной строке.

```
dplyr::glimpse(super_sleepers)
```

```
Rows: 4
Columns: 4
$ rating          <int> 1, 2, 3, 4
$ animal          <chr> "коала", "ёж", "ленивец", "панда"
$ country         <chr> "Австралия", "Италия", "Перу", "Китай"
$ avg_sleep_hours <dbl> 21, 18, 17, 10
```

`dplyr::` — вызов функции из пакета `dplyr`. Пакет должен быть установлен.

СВОДНЫЕ СТАТИСТИКИ: `summary()`

```
summary(super_sleepers["avg_sleep_hours"])
```

```
 avg_sleep_hours
Min.   :10.00
1st Qu.:15.25
Median :17.50
Mean   :16.50
3rd Qu.:18.75
Max.   :21.00
```

Min./Max. — минимум/максимум; Median/Mean — медиана/среднее;
1st Qu./3rd Qu. — первый/третий квартили.

Имена строк и столбцов

ИМЕНА СТРОК

Аргумент `row.names` задаёт имена строк при создании таблицы.

```
super_sleepers <- data.frame(
  rating, animal, country, avg_sleep_hours,
  row.names = c("row_1", "row_2", "row_3", "row_4")
)
super_sleepers
```

```
      rating  animal   country avg_sleep_hours
row_1      1   коала Австралия              21
row_2      2      ёж    Италия              18
row_3      3 ленивец      Перу              17
row_4      4   панда     Китай              10
```

ПРОВЕРКА ИМЁН СТОЛБЦОВ: `check.names`

`TRUE` гарантирует допустимые и уникальные имена (по умолчанию).
`FALSE` — сохраняет исходные имена, включая повторы и пробелы.

```
super_sleepers <- data.frame(
  animal = rating, animal = animal,
  country, avg_sleep_hours,
  check.names = TRUE
)
names(super_sleepers)[1:2]
```

```
[1] "animal"   "animal.1"
```

```
df_raw <- data.frame(
  animal = rating, animal = animal,
  country, avg_sleep_hours,
  check.names = FALSE
)
names(df_raw)[1:2]
```

```
[1] "animal" "animal"
```

`check.names = TRUE` исправит `"avg sleep"` на `avg.sleep` без ошибки.
Без кавычек или обратных кавычек пробел в имени вызовет ошибку синтаксиса.

ИМЕНА СТРОК — УНИКАЛЬНЫ

Если дважды указать `row_1`, R сообщит об ошибке.

```
super_sleepers <- data.frame(
  rating, animal, country, avg_sleep_hours,
  row.names = c("row_1", "row_1", "row_3", "row_4")
)
```

```
Error: duplicate row.names: row_1
```

ПЕРЕИМЕНОВАНИЕ СТОЛБЦОВ

```
names(super_sleepers) <- c("col_1", "col_2", "col_3", "col_4")
super_sleepers
```

```
  col_1   col_2     col_3 col_4
1     1   коала Австралия    21
2     2      ёж    Италия    18
3     3 ленивец      Перу    17
4     4   панда     Китай    10
```

Таблица, матрица и список

ИЗ МАТРИЦЫ В ТАБЛИЦУ

Каждый столбец матрицы становится столбцом таблицы.

```
my_matrix <- matrix(
  c(1, 2, 3, 4, 5, 6), nrow = 2
)
my_matrix
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
df_from_matrix <- data.frame(my_matrix)
df_from_matrix
```

```
  X1 X2 X3
1  1  3  5
2  2  4  6
```

Если имён столбцов нет, R создаёт `X1`, `X2`, `X3`.

ИЗ СПИСКА В ТАБЛИЦУ

`data.frame()` также принимает список векторов одинаковой длины.

```
my_list <- list(rating = rating, animal = animal,
                country = country,
                avg_sleep_hours = avg_sleep_hours)
super_sleepers <- data.frame(my_list)
super_sleepers
```

```
  rating  animal   country avg_sleep_hours
1      1   коала Австралия              21
2      2      ёж    Италия              18
3      3 ленивец      Перу              17
4      4   панда     Китай              10
```

`as.matrix()`: ИЗ ТАБЛИЦЫ В МАТРИЦУ

Выберем числовые столбцы и преобразуем их в матрицу.
Все элементы матрицы будут одного типа.

```
cols <- c("rating", "avg_sleep_hours")
M <- as.matrix(super_sleepers[cols])
M
typeof(M)
```

```
     rating avg_sleep_hours
[1,]      1               21
[2,]      2               18
[3,]      3               17
[4,]      4               10

[1] "double"
```

```
typeof(as.matrix(super_sleepers))
```

```
[1] "character"
```

Если преобразовать всю таблицу с текстовыми столбцами, числа станут текстом.

Объединение таблиц

`cbind()`: ДОБАВЛЕНИЕ СТОЛБЦОВ

Объединим признаки из двух таблиц с помощью `cbind()`.

```
super_sleepers_1 <- data.frame(
  rating = 1:4, animal = c("коала", "ёж", "ленивец", "панда"))
super_sleepers_2 <- data.frame(
  country = c("Австралия", "Италия", "Перу", "Китай"),
  avg_sleep_hours = c(21, 18, 17, 10))
# одинаковые наблюдения должны идти в одном порядке
super_sleepers <- cbind(super_sleepers_1, super_sleepers_2)
super_sleepers
```

```
  rating  animal   country avg_sleep_hours
1      1   коала Австралия              21
2      2      ёж    Италия              18
3      3 ленивец      Перу              17
4      4   панда     Китай              10
```

`rbind()`: ДОБАВЛЕНИЕ СТРОК

```
super_sleepers_1 <- data.frame(
  rating = 1:2,
  animal = c("коала", "ёж"),
  country = c("Австралия",
              "Италия"),
  avg_sleep_hours = c(21, 18)
)
```

```
super_sleepers_2 <- data.frame(
  rating = 3:4,
  animal = c("ленивец", "панда"),
  country = c("Перу", "Китай"),
  avg_sleep_hours = c(17, 10)
)
```

```
super_sleepers <- rbind(
  super_sleepers_1,
  super_sleepers_2
)
super_sleepers
```

```
  rating  animal   country avg_sleep_hours
1      1   коала Австралия              21
2      2      ёж    Италия              18
3      3 ленивец      Перу              17
4      4   панда     Китай              10
```

Создание пустой таблицы

ПУСТАЯ ТАБЛИЦА: ЗАДАЁМ ТИПЫ СТОЛБЦОВ

Пустая таблица содержит имена и типы столбцов, но не содержит строк.

```
super_sleepers_empty <- data.frame(
  rating = integer(),                # место в рейтинге
  animal = character(),              # животное
  country = character(),             # страна
  avg_sleep_hours = numeric()        # среднее время сна, ч/сутки
)
super_sleepers_empty
```

```
[1] rating          animal          country         avg_sleep_hours
<0 rows> (or 0-length row.names)
```

`integer()`, `numeric()` и `character()` создают векторы длины 0 нужного типа.

ПРОВЕРКА ТИПОВ В ПУСТОЙ ТАБЛИЦЕ

`str()` показывает: наблюдений нет, но четыре столбца уже определены.

```
str(super_sleepers_empty)
```

```
'data.frame':   0 obs. of  4 variables:
 $ rating         : int
 $ animal         : chr
 $ country        : chr
 $ avg_sleep_hours: num
```

`integer()`, `numeric()` и `character()` создают векторы длины 0 нужного типа.

ПУСТАЯ ТАБЛИЦА ИЗ ГОТОВОГО ОБРАЗЦА

Выберем ноль строк и все столбцы готовой таблицы.
Имена и типы столбцов сохранятся.

```
super_sleepers_empty <- super_sleepers[FALSE, ]
super_sleepers_empty
str(super_sleepers_empty)
```

```
[1] rating          animal          country         avg_sleep_hours
<0 rows> (or 0-length row.names)

'data.frame':   0 obs. of  4 variables:
 $ rating         : int
 $ animal         : chr
 $ country        : chr
 $ avg_sleep_hours: num
```

ДОБАВЛЕНИЕ НАБЛЮДЕНИЯ В ПУСТУЮ ТАБЛИЦУ

Создадим строку с теми же именами столбцов.
`rbind()` добавит её к пустой таблице.

```
new_row <- data.frame(
  rating = 1L,  # целое число
  animal = "коала",
  country = "Австралия",
  avg_sleep_hours = 21
)
super_sleepers_filled <- rbind(super_sleepers_empty, new_row)
super_sleepers_filled
```

```
  rating animal   country avg_sleep_hours
1      1  коала Австралия              21
```

СТОЛБЕЦ: ВЕКТОР ИЛИ ТАБЛИЦА

Сравним извлечение одного столбца разными способами.

```
df <- super_sleepers
```

```
x <- df$animal
x
class(x)
```

```
[1] "коала"
[2] "ёж"
[3] "ленивец"
[4] "панда"

[1] "character"
```

```
x <- df[["animal"]]
x
class(x)
```

```
[1] "коала"
[2] "ёж"
[3] "ленивец"
[4] "панда"

[1] "character"
```

```
x <- df["animal"]
x
class(x)
```

```
   animal
1   коала
2      ёж
3 ленивец
4   панда

[1] "data.frame"
```

ВЫБОР СТРОК И СТОЛБЦОВ

Слева от запятой — строки, справа — столбцы.
`drop = FALSE` сохраняет таблицу при выборе одного столбца.

```
df[1:2,
   c("animal", "avg_sleep_hours")]
```

```
  animal avg_sleep_hours
1  коала              21
2     ёж              18
```

```
df[1:2, "avg_sleep_hours"]
```

```
[1] 21 18
```

```
df[1:2, "avg_sleep_hours",
   drop = FALSE]
```

```
  avg_sleep_hours
1              21
2              18
```

СОЗДАНИЕ ВЫЧИСЛЯЕМОГО СТОЛБЦА

Часы бодрствования — разница между 24 часами и временем сна.

```
df$awake_hours <- 24 - df$avg_sleep_hours
df[c("animal", "avg_sleep_hours", "awake_hours")]
```

```
   animal avg_sleep_hours awake_hours
1   коала              21           3
2      ёж              18           6
3 ленивец              17           7
4   панда              10          14
```

ВЫБОР СТРОК ПО УСЛОВИЮ

Оставим животных, которые спят не менее 18 часов в сутки.

```
keep <- df$avg_sleep_hours >= 18
keep
```

```
[1]  TRUE  TRUE FALSE FALSE
```

```
selected <- df[keep, ]
selected[c("animal", "avg_sleep_hours")]
nrow(selected)  # число строк
```

```
  animal avg_sleep_hours
1  коала              21
2     ёж              18

[1] 2
```

СОРТИРОВКА СТРОК: `order()`

`order()` возвращает индексы строк по возрастанию времени сна. `decreasing = TRUE` задаёт порядок по убыванию.

```
idx <- order(df$avg_sleep_hours)
ranked <- df[idx, ]
ranked[c("animal", "avg_sleep_hours")]
```

```
   animal avg_sleep_hours
4   панда              10
3 ленивец              17
2      ёж              18
1   коала              21
```

```
idx <- order(df$avg_sleep_hours,
             decreasing = TRUE)
ranked <- df[idx, ]
ranked[c("animal", "avg_sleep_hours")]
```

```
   animal avg_sleep_hours
1   коала              21
2      ёж              18
3 ленивец              17
4   панда              10
```

ПРОПУСКИ В ТАБЛИЦЕ

Неизвестная страна не мешает использовать известное время сна.

```
sleep_missing <- super_sleepers
sleep_missing$avg_sleep_hours[4] <- NA
sleep_missing$country[3] <- NA
hours <- sleep_missing$avg_sleep_hours
```

```
sum(!is.na(hours))  # известно время сна
```

```
[1] 3
```

```
sum(complete.cases(sleep_missing))  # полные строки
```

```
[1] 2
```

```
mean(hours, na.rm = TRUE)  # не учитываем NA
```

```
[1] 18.66667
```

Факторы

ФАКТОР: ЗНАЧЕНИЯ И УРОВНИ

Фактор хранит значения наблюдений и список категорий — уровни.
Класс `factor` обозначает вид объекта в R, а не категорию животного.

```
animal <- factor(
  c("коала", "ёж", "коала", "ёж"),
  levels = c("коала", "ёж", "ленивец")
)
animal
```

```
[1] коала ёж    коала ёж
Levels: коала ёж ленивец
```

```
class(animal)   # вид объекта в R
levels(animal)  # список категорий
```

```
[1] "factor"

[1] "коала"   "ёж"
[3] "ленивец"
```

`data.frame()` и `table()`

`data.frame()` собирает векторы в таблицу.
`table()` подсчитывает, сколько раз встречается каждая категория.

Таблица наблюдений

```
data.frame(animal)
```

```
  animal
1  коала
2     ёж
3  коала
4     ёж
```

Подсчёт частот

```
counts <- table(animal)
counts
```

```
animal
  коала       ёж ленивец
      2        2        0
```

Частоты в `data.frame`

```
# Freq — число записей
as.data.frame(counts)
```

```
    animal Freq
1   коала    2
2      ёж    2
3 ленивец    0
```

Класс результата `table()` — `table`; `as.data.frame()` преобразует его в `data.frame`.

ЧАСТОТЫ И НЕИСПОЛЬЗУЕМЫЕ УРОВНИ

После выборки уровни сохраняются, даже если записей с ними нет.

```
table(animal)  # число записей
```

```
animal
  коала       ёж ленивец
      2         2         0
```

```
koala <- animal[animal == "коала"]
table(koala)
```

```
koala
  коала       ёж ленивец
      2         0         0
```

```
# удаляем неиспользуемые уровни
koala <- droplevels(koala)
table(koala)
```

```
koala
коала
    2
```

ПОРЯДОК И НАЗВАНИЯ УРОВНЕЙ

Вверху — показываем ежа первым в таблице частот.
Внизу — переименовываем категории в копии исходного `animal`.

```
animal_order <- factor(animal,
  levels = c("ёж", "коала", "ленивец"))
table(animal_order)
```

```
animal_order
     ёж   коала ленивец
      2       2       0
```

```
animal_labels <- animal
levels(animal_labels) <- c(
  "коалы",   # вместо "коала"
  "ежи",     # вместо "ёж"
  "ленивцы") # вместо "ленивец"
table(animal_labels)
```

```
animal_labels
  коалы     ежи ленивцы
      2       2       0
```

При `levels(...) <- c(...)` первое имя заменяет первый уровень, второе — второй и т. д.

УПОРЯДОЧЕННЫЙ ФАКТОР

У размеров перчаток есть смысловой порядок S < M < L.
`ordered = TRUE` позволяет сравнивать категории по этому порядку.

```
size <- factor(
  c("M", "S", "L"),
  levels = c("S", "M", "L"),
  ordered = TRUE)
size  # значения остаются M, S, L
```

```
[1] M S L
Levels: S < M < L
```

```
size > "S"   # сравнение по порядку
class(size)  # ordered — тоже factor
```

```
[1]  TRUE FALSE  TRUE

[1] "ordered" "factor"
```

Обычный фактор: `==`, `!=`; упорядоченный — также `<`, `>`, `<=`, `>=`.

ЗНАЧЕНИЯ ВНЕ СПИСКА УРОВНЕЙ

В `factor()` значение вне заданных уровней становится `NA`.
Исходное написание сохраняем в `raw`, чтобы проверить его.

```
raw <- c("коала", "еж", "ёж")
allowed <- levels(animal) # категории
checked <- factor(raw, levels = allowed)
checked
```

```
[1] коала <NA>  ёж
Levels: коала ёж ленивец
```

```
raw[is.na(checked)]  # исходное написание
```

```
[1] "еж"
```

```
raw[raw == "еж"] <- "ёж"
checked <- factor(raw, levels = allowed)
checked  # исправленный фактор
```

```
[1] коала ёж    ёж
Levels: коала ёж ленивец
```

КОДЫ ФАКТОРА И ТЕКСТОВЫЕ ЗНАЧЕНИЯ

Исходный `animal`: коала → 1, ёж → 2, ленивец → 3.
Код — позиция уровня; это не числовое измерение.

```
class(animal)   # класс объекта
typeof(animal)  # внутреннее хранение
```

```
[1] "factor"

[1] "integer"
```

```
as.numeric(animal)  # коды уровней
```

```
[1] 1 2 1 2
```

```
as.character(animal)  # подписи
```

```
[1] "коала" "ёж"    "коала"
[4] "ёж"
```

ЧАСТОТЫ СОЧЕТАНИЙ: `table()`

Новая таблица: строка — запись наблюдателя о времени сна.
В столбце `animal` зададим два уровня: коала и ёж.

```
observations <- data.frame(
  animal = factor(c("коала", "ёж", "коала", "ёж"),
                  levels = c("коала", "ёж")),
  observer = c("Анна", "Анна", "Анна", "Борис"),  # наблюдатель
  sleep_hours = c(21, 18, 20, 17)                 # ч/сутки; для среднего
)
```

```
table(observations$animal,    # строки
      observations$observer)  # столбцы
```

```
        Анна Борис
  коала    2     0
  ёж       1     1
```

У Анны 2 записи о коале, у Бориса — 0. Считаем записи, не часы сна.

СРЕДНЕЕ ПО ГРУППАМ: `tapply()`

Группируем записи по животному, независимо от наблюдателя.
Результат — одномерный массив средних, `observations` не меняется.

```
observations
```

```
  animal observer sleep_hours
1  коала     Анна           21
2     ёж     Анна           18
3  коала     Анна           20
4     ёж    Борис           17
```

```
tapply(observations$sleep_hours,  # значения
       observations$animal,       # группы
       mean)                      # функция
```

```
коала    ёж
 20.5  17.5
```

Коала: (21 + 20) / 2 = 20.5; ёж: (18 + 17) / 2 = 17.5 ч/сутки.
Передаём `mean` без скобок: `tapply()` вызовет её для каждой группы.

ФАКТОР ПОСЛЕ СОХРАНЕНИЯ В CSV

Из CSV читаем названия животных как текст, затем создаём фактор.

```
to_save <- data.frame(animal = animal)
write.csv(to_save, "animals.csv",
  row.names = FALSE) # без имён строк
restored <- read.csv("animals.csv")
class(restored$animal)  # теперь это текст
```

```
[1] "character"
```

```
restored$animal <- factor(
  restored$animal,
  levels = c("коала", "ёж", "ленивец"))
restored$animal
```

```
[1] коала ёж    коала ёж
Levels: коала ёж ленивец
```

Список категорий задаём заново: он не хранится в CSV.
Для сохранения самого R-объекта: `saveRDS()` / `readRDS()`.